Reinforcement Learning Agent for Legend of Prince Valiant (NES)

Name	*Abdul Samad*
Submission date	*16 October 2026*
GitHub	***https://github.com/Abdulsamad61/Legend-of-Prince-Valiant***
Video	***https://drive.google.com/file/d/1wUEoE5jqj3PNdKdhHESkAeVrdx_ytJhn/view?usp=sharing***

Problem statement

The task is to train an agent that clears Levels 1-3 of Legend of Prince Valiant (NES) from the game screen, in at least 7 of 10 runs.

Solution approach

In this work, initially, I wrapped the NES game in a Gymnasium environment (stable-retro), tried PPO from pixels first, and then moved to a search-then-imitate approach. In this task, PPO alone would not get through a level. Imitation learning, provides the opportunity to solve the problem by using the copying examples instead of by trial and error. finds one route that clears the level. It is a PPO-style policy network then learns to reproduce that route from the screen. The final agents clear every level in 10 out of 10 evaluation episodes with deterministic actions. Levels always evaluated from the game's normal start. presents the results of both approaches evaluated in this task. Imitation learning achieved the best overall performance, successfully clearing the levels, whereas PPO initially showed promising progress but ultimately failed to complete them.

Table 1 presents the results of both approaches evaluated in this task. Imitation learning achieved the best overall performance, successfully clearing the levels, whereas PPO initially showed promising progress but ultimately failed to complete them.

Table 1 Completion rate (10 episodes): imitation agents vs. PPO baseline.

Level	**Final agent**	**Deterministic**	**Temp. 0.1**	**Temp. 0.5**	**PPO baseline**
1-The Marsh	imitation	**10/10**	8/10	3/10	0/10
2-The Forest	imitation	**10/10**	10/10	10/10	0/10
3-The Viking	imitation	**10/10**	10/10	10/10	not run

Setup

In Table 2 shows the environment setup, presenting how the game screen is processed, which button combinations the agent can press on each level, where each level starts, which rules were followed, and the random seed.

Table 2 Setup Settings.

Item	Setting
Observation	frame skip 4, grayscale, 84×84, stack of 4 frames. The imitation agents get one extra channel with the step number in binary.
Actions	A reduced set of button combinations is used in Levels 1 to 3. These buttons cover moving, jumping, shooting and ducking.
Starts	Level 1: normal game start. Levels 2 and 3: a save state at the first frame of that level.
Rules	lives, timer and invincibility are never changed; no level skips; savestates only used for training and route search.
Seed	0 for every run; the emulator itself is deterministic.

Algorithm choice

PPO (Stable-Baselines3, CnnPolicy) was my starting point. It is stable on pixel input, uses parallel emulators easily, fits a small discrete action set. In this work, same network design was used for the imitation agents, but trained them separately from scratch, so the two methods can be compared fairly.

In imitation based learning, game levels require thousands of sequential decisions, precise timing, and coordinated actions to overcome obstacles and defeat enemies.

Reward design

The reward is only used by PPO (and as a progress score in the video and graphs). The imitation agents are trained with a supervised loss and do not use it. PPO was not trained on Level 3 because it already failed on Levels 1-2. Table 3 and Table 4 shows the rewards used in this work. The imitation agents do not learn from the reward because the imitation agents use **behaviour cloning**. A **Stable-Baselines3 CnnPolicy** network learns from the **demonstration route** (screen + step number and key).

Table 3 Rewards used for Level 1

Reward term	**Weight**
New furthest progress	+0.1 per pixel
New section reached	+20
Score increase	+0.01 per point
Boss hit	+1 per hit
Level cleared	+50
Life lost	−5
Time penalty	−0.001 per step

Table 4 Rewards used for Level 2

Reward term	**Weight**
Robber shot	+3 per robber
New distance walked	+0.25 per unit
Energy gained / lost	±0.01 per point
Aim bonus (crosshair closer to / further from nearest robber)	+0.01 / −0.01 per pixel
Level cleared	+50
Life lost	−3
Time penalty	−0.001 per step

Exploration to exploitation

PPO explores through its entropy bonus (ent_coef 0.02) and the randomness of its own policy; the learning rate decays linearly from $2.5 \cdot 10^{-4}$ to 0 and the clip range is 0.1. I follow the shift by looking at the policy entropy: maximum entropy means random key presses, falling entropy means the agent is committing to actions.

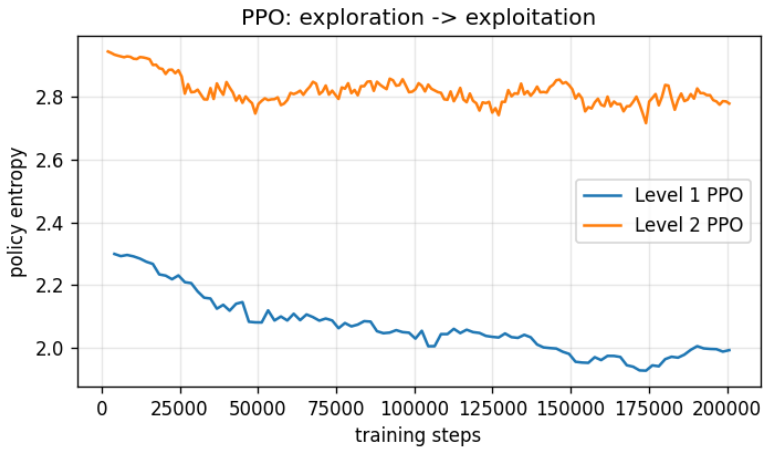


Figure 1 PPO policy entropy over 200k training steps (Colab run).

- **Level 1:** entropy starts at 2.3 (fully random with 10 actions) and drops to about 2.0 within the first ~50k steps, then flattens. The agent starts preferring some actions but stays quite random.
- **Level 2:** entropy stays near 2.8 (the maximum with 19 actions is 2.94). After 200k steps it is still almost entirely exploring.
- In both cases the shift never really completes, which matches the reward curves: PPO never finds the rewards that would make it commit.
- The imitation agents have no exploration phase: they are trained supervised and evaluated with deterministic actions.

Why the Colab runs are only 200k steps. Longer PPO runs in development (2-core CPU, 0.5-1.5M steps, including versions that started from behaviour cloning) never cleared a level. The best reached section 2 of Level 1 in 3 of 10 runs, and Level 2 PPO hit about 5 robbers per life as shown in Figure 2 . Since 7× more steps did not change the outcome, the Colab baseline uses 200k steps per level.

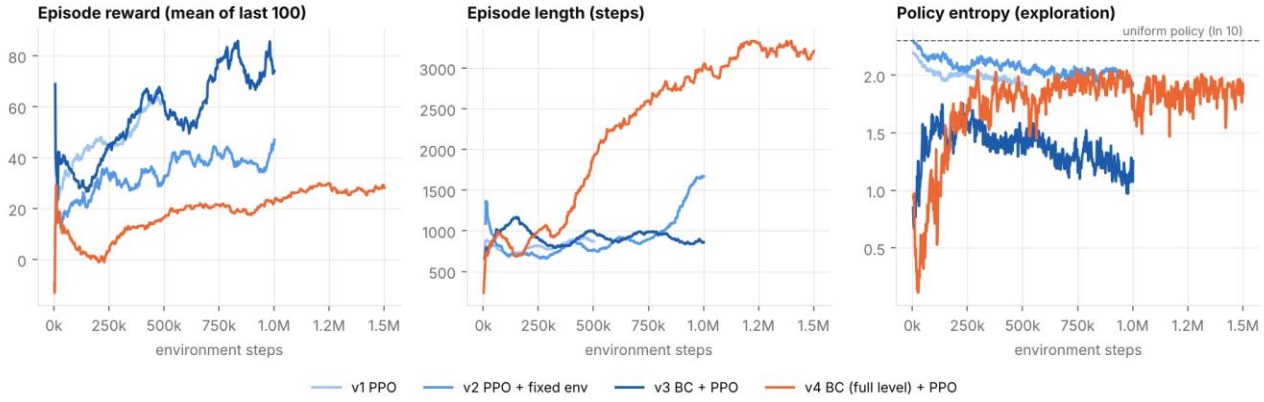


Figure 2 Longer Level 1 PPO runs (0.5–1.5M steps, v1–v4). None cleared the level.

Hurdles

Table 5 Problems and solutions.

Problem	What I did
The built-in integration only gives lives and score.	Found position, section/room, timer, boss health and quest flags in RAM by probing.
x is a 16-bit value; using only the low byte made progress jump around.	Read both bytes: x = RAM[86] + 256·RAM[87].
PPO never got past the first part of Level 1 in my development runs (0.5–1.5M steps)	Switched to finding a route with a savestate search and learning it by imitation.
Some frames on the route look identical but need different keys (timed waits).	Added a channel with the step number in binary, so the policy knows *when* it is (14 bits for the long Level 3 route).
Level 3: a 10:00 timer for the whole maze; my first full route ran out of time halfway.	Rebuilt the route room by room, searching each room for the fastest exit, and collected the four T items (+60 s each). It now finishes with 0:56 left.
Re-running training into the same folder mixed checkpoints from two runs in my graphs.	Delete the run folder before each training run.

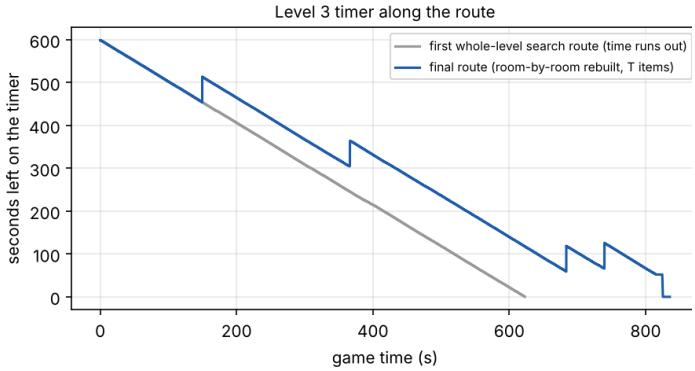


Figure 3 Level 3 timer along the first full search route (runs out) and along the rebuilt room-by-room route (jumps are T items).

Results

For every saved checkpoint I play one episode with deterministic actions and record the episode reward, the episode length and whether the level was cleared (make_graphs.py). PPO is plotted against environment steps, imitation against training samples (epochs × route length). One episode per checkpoint is enough because the game and the actions are deterministic.

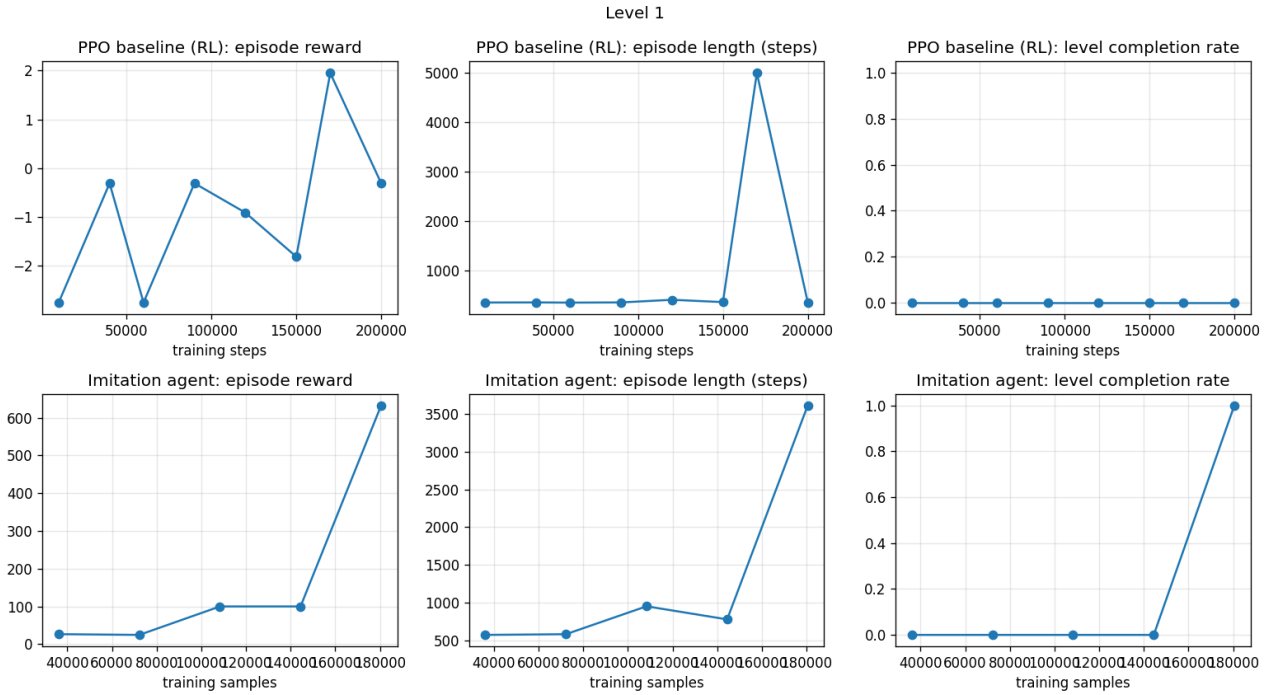


Figure 4 Level 1. Top: PPO, 200k steps; reward stays between −3 and +2 and the level is never cleared. Imitation clears the level at epoch 50.

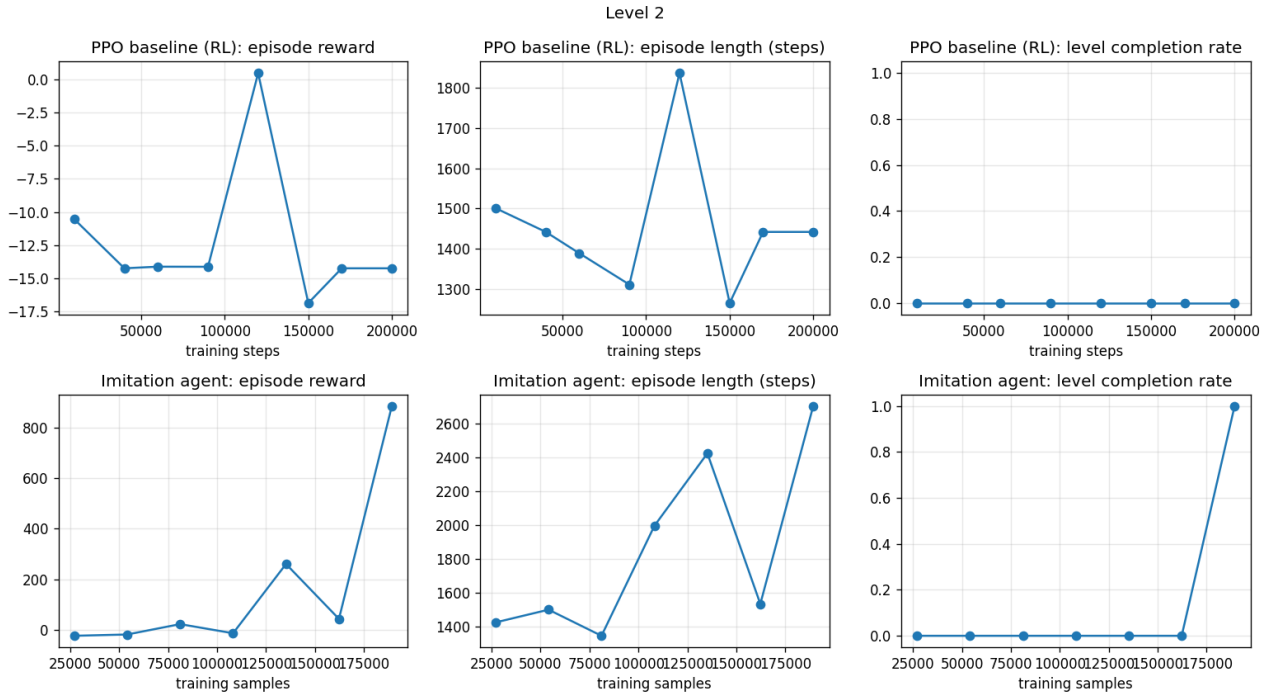


Figure 5 Level 2. PPO stays around -14 reward and never clears. Imitation clears the level at epoch 70.

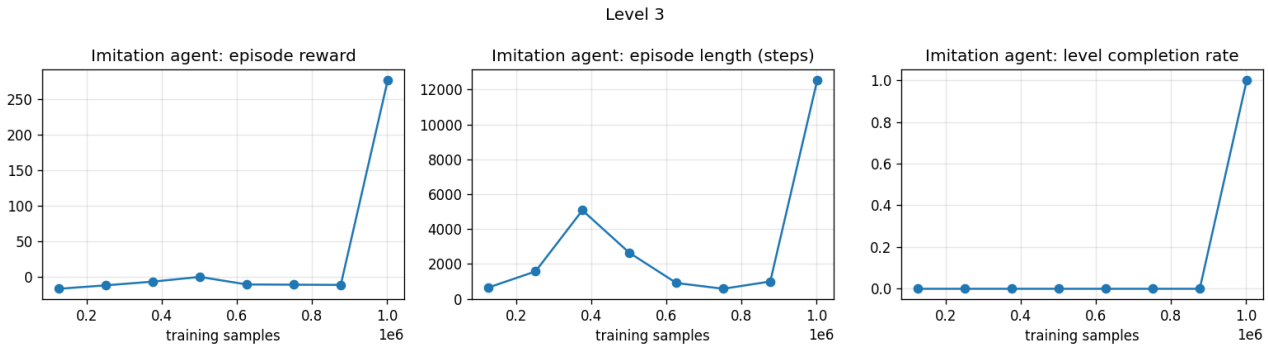


Figure 6 Level 3 (imitation only). The agent clears the level at epoch 80.

Final evaluation

10 episodes per test with the final agent of each level (eval.py). Deterministic actions are the task criterion (≥ 7/10).

Test	Level 1	Level 2	Level 3
Deterministic	**10/10**	**10/10**	**10/10**
Temperature 0.1	8/10	10/10	10/10
Temperature 0.5	3/10	10/10	10/10
PPO baseline (deterministic)	0/10	0/10	not run

Every cleared episode keeps all 4 lives and follows the learned route exactly (Level 1: 3,611 steps, Level 2: 2,704, Level 3: 12,533). With sampled actions Level 1 is the most fragile: at temperature 0.1 it still clears 8/10, but at 0.5 only 3/10, because a single wrong key at a timing-critical spot (mostly in section 3) costs the run. Levels 2 and 3 stay at 10/10 because their policies are very confident at every step. The PPO baselines end at the first obstacle (Level 1: section 1, reward -0.3; Level 2: no robbers hit).

Training time and hardware

Item	Value
Hardware (Google Colab)	AMD EPYC 9B45, 48 CPU cores, 176 GB RAM, NVIDIA RTX PRO 6000 Blackwell GPU
Software	Python 3.13.15 · stable-retro 1.0.1 · stable-baselines3 2.9.0 · gymnasium 1.4.0 · torch 2.11.0+cu130 · numpy 2.1.3 · seed 0
PPO Level 1 (200k steps, 8 envs)	1.5 min (87 s)
PPO Level 2 (200k steps, 8 envs)	1.7 min (99 s)
Imitation Level 1 / 2 / 3	0.7 min / 1.1 min / 2.3 min (50 / 70 / 80 epochs)
Total on Colab	**≈ 7.5 min of training** (PPO 3.2 min + imitation 4.1 min)
Development, before Colab (2-core CPU, 8 GB RAM, no GPU)	PPO experiments ≈ 7.5 h (Level 1 v1–v4: 4M steps ≈ 5 h; Level 2: 1.53M steps ≈ 2.5 h) · route search ≈ 35 h (Level 1 ≈ 7 h, Level 3 ≈ 28 h). The routes are in the repo (optional_demo/), so the Colab run starts from them.
Total compute for the project	**≈ 43 hours** (almost all of it route search and the long PPO experiments)

Comparision of algorithms

PPO and imitation are compared on the same levels with the same network, observation and evaluation (Levels 1 and 2, Figure 4 to Figure 5 and the evaluation table): PPO 0/10 on both, imitation 10/10 on both.

Reproducing

Open colab_run.ipynb and run the cells top to bottom. Main commands per level: train.py / train_l2.py (PPO), bc_exact.py / bc_exact_l2.py / bc_exact_l3.py (imitation), eval.py (10-episode evaluation), make_graphs.py (figures) and make_video_all.py (video).

The game ROM "Legend of Prince Valiant (NES)" is not included in this repository because it is copyrighted. Which can be downloaded.